AOSP 进阶丛书 · 第十二册 · ANDROID 13 (API 33)

媒体管线
音频与相机

AudioFlinger 混音车间与 AudioPolicy 调度室、三档延迟与焦点约定、Camera2 请求-结果流水线与 ZSL/多摄、MediaCodec 双队列与 Codec2、MediaDrm 与 AAudio。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：媒体是三条"实时流水线" 2
第一章 音频管线：从 write() 到扬声器 2
第二章 相机管线：请求-结果流水线 4
第三章 编解码与 DRM：双队列模型 5
第四章 设计决策九问 6
附录 A 观测实操 7
附录 B 快问快答（14 条） 7
附录 C 自检题 8
第五章 音频框架的前世与生态位 8
附录 D 动手实验室（半天量） 8
教科书习题集（分级） 9
术语表 9
排障手册：媒体管线问题定位 9

本册速览

音频：AF 混音车间+AP 调度室 · 三档延迟(normal/fast/MMAP)
焦点是约定：策略裁决不拦混音 · Offload 省电与线程家族
相机：Camera2 请求-结果流水线 · ZSL/多摄/逻辑相机
编解码：MediaCodec 双队列+Codec2 · MediaDrm L1/L3 与 AAudio
附录 D 五实验室：延迟实测/路由现场/焦点攻防/in-flight/Offload

前言：媒体是三条"实时流水线"

媒体子系统的共性是**实时性**：音频按周期混音（20ms 级）、相机按帧出图（33ms 级）、编解码按 buffer 流转——它们都不是"请求-响应"而是"流水线+节奏"。本册讲三条管线：**音频**（AudioFlinger 混音车间 + AudioPolicy 调度室）、**相机**（Camera2 请求-结果流水线）、**编解码**（MediaCodec/Codec2 双队列），末章 MediaDrm 与书11 信任链衔接。

源码基线不变（本树 Android 13）。主角目录：`frameworks/av/services/audioflinger`（AF）、`frameworks/av/services/audiopolicy`（AP）、`frameworks/av/services/camera/libcameraservice`（CameraService）、`hardware/interfaces/{audio,camera}`（HAL）。

第一章 音频管线：从 write() 到扬声器



图 1 音频四段：AudioTrack→AudioFlinger 混音→AudioPolicy 策略→HAL/DAC，含延迟三档位与焦点语义

1.1 逐段注释（真实锚点）

cpp frameworks/av（骨架+讲解注释）

```
// App 侧：AudioTrack::set（media/libaudioclient/AudioTrack.cpp:500/531/849）
// 讲解注释①：set 声明格式（采样率/声道/usage），随后 write()
// 写入环形 buffer；FAST track 则共享内存直投（低延迟档）。
// 服务端：AudioFlinger::createTrack（audioflinger/AudioFlinger.cpp:1023）
// 讲解注释②：AF（:306 构造，audioserver 进程）先问 AP：
```

```
// getOutputForAttr ——这个流（usage=media/通话/铃）该去哪个设备?
// 讲解注释③：MixerThread::threadLoop_write（Threads.cpp:4908）：
// 每周期把该输出上所有 Track 混音写 HAL；
// DirectOutputThread（:2827）不混直通（高音质流）。
// 策略侧：AudioPolicyManager（audiopolicy/manager/）
// 讲解注释④：路由（插耳机切换）/音量（按 stream type）/焦点（互抢裁决）
// ——AP 只决策，AF 执行；两者进程内协作、接口分离。
```

1.3 AudioFlinger 的线程家族（读码地图）

text Threads.cpp 的线程谱系（audioserver 内）

```
PlaybackThread // 普通回放母类：周期=frameCount/sampleRate
├─ MixerThread(:4908) // 混音：多 Track→一路 PCM→HAL
│  └─ OffloadThread // 压缩流（MP3/AAC）不解码直送 DSP——省电主力
└─ DirectOutputThread(:2827) // 不混直通（高音质/低延迟单流）
RecordThread // 录音：反向 PCM 从 HAL 取
MmapThread // 8+：MMAP 共享内存直投（AAudio 低延迟的底座）
// 讲解注释：每"输出设备+打开配置"组合一条线程——
// 蓝牙+扬声器同时出声=两条 MixerThread 并行；dumpsys af 逐线程可见。
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释媒体管线的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

设计洞察 · 逐段精读：Offload 为什么省电

普通路径：MP3→CPU 解码→PCM→混音→HAL（CPU 全程醒）。Offload：压缩流直接交 DSP，CPU 可睡——长音乐播放功耗差 30%+。代价：无法混音（独占输出）+seek 延迟——所以"通知声叠在音乐上"时策略层会切回普通路径，这个切换本身也是 AP 的决策。

1.4 音量与路由：策略层的两张表

text AudioPolicyManager 的决策素材（读码归纳）

```
音量表：STREAM_(MUSIC/RING/ALARM/VOICE_CALL/…) × 设备(扬声器/耳机/BT)
→ setStreamVolume 只改表；真实音量在 AudioFlinger 施加
（软件音量在混音时乘增益；硬件音量直写 HAL amplifier）
路由表：设备状态机（插拔/连接 BT 触发重路由）
→ 每流 usage→设备 的映射规则（引擎可配：engineconfigurable/）
→ "导航说话时音乐闪避（duck）"= 策略引擎的注意力仲裁
// 讲解注释：dumpsys media.audio_policy 的前两段就是这两张表的实时投影。
```

设计洞察 · 逐段精读：三个延迟档与选型

normal（binder 写+混音周期，100ms 级）→ fast track（共享内存+低周期 ~20ms）→ **MMAP no-IRQ**（8+：硬件时钟直驱 <10ms，AAudio 首选）。音游/通话回声消除选 MMAP；普通提示音 normal 足矣——延迟是用功耗换的，不是免费午餐。

1.2 焦点：约定而非强制

App 的 requestAudioFocus 是**约定**：策略侧裁决并广播 LOSS/GAIN，但混音不物理停流——"被夺焦后停音乐"是 App 守约的行为。系统只在铃声/通话场景强制降权（duckt）。面试高频："两个 App 同时放歌会怎样"——都在放（物理层不拦），体验层靠焦点协商（所以流氓 App 能"抢音"）。

第二章 相机管线：请求-结果流水线



图 2 Camera2 四段：openCamera(:825)→CameraService connectHelper(:1750)→HAL 3A/ISP→onCaptureCompleted，Camera1/2/X 对照与权限隐私

2.1 逐段注释（真实锚点）

java CameraManager/CameraService（骨架+讲解注释）

```
// App：CameraManager.openCamera（core/.../CameraManager.java:825/862/927）
// 讲解注释①：异步 open——回调拿到 CameraDevice；
// createCaptureSession 绑定输出 Surface（预览/拍照/分析各自流）。
// 服务端：CameraService（libcameraservice/CameraService.cpp:145 构造）
```

```
// 讲解注释②：connectHelper（:1750）：客户端接入仲裁——
// 高优客户端（SystemUI 人脸）可踢低优；权限与"前台可见"检查。
// 讲解注释③：请求流水线：setRepeatingRequest（预览=无限重复）+
// capture（拍照=插一个高分辨率请求）；按 HAL in-flight 节奏逐个下发。
// HAL：hardware/interfaces/camera/provider/（2.4-2.6，AIDL 化进行中）
// 讲解注释④：3A（AE/AF/AWB）与 ISP 在厂商 HAL；每帧回
// TotalCaptureResult（图像+metadata"怎么拍的"）——
// 异步请求-结果对是 Camera2 的核心抽象。
```

2.3 多摄与 ZSL（Camera2 能力的兑现）

text 为什么只有流水线架构能做（读码归纳）

```
ZSL 零快门：环形缓冲持续捕帧，按快门时回溯取帧——
需要逐帧控制（新旧架构里 Camera1 做不到，只能"提前开预览"）
多摄同步：逻辑相机（ID 背后多个物理头）同一请求下发
——深度/虚化=两路帧时间戳对齐（HAL 保证 sensor sync）
HDR+：短曝多帧+算法合成——逐帧改曝光参数的流水线是前提
// 讲解注释：CameraManager.getCameraIdList 里带 LOGICAL_MULTI_CAMERA
// 能力集的 ID 即逻辑相机；物理头经 getPhysicalCameraIds 暴露。
```

设计洞察 · 逐段精读：Camera1/2/X 一句话

Camera1 黑箱（takePicture 一锤子）；Camera2 把 ISP 开成**可编程请求队列**（逐帧改参数/多 Surface/ZSL/RAW）；CameraX 是 Camera2 的易用层（生命周期感知），不是新管线。

2.2 排障三查

`dumpsys media.camera`（客户端/会话/in-flight 全表）；预览黑屏查三处：会话 Surface 配置、流组合是否被 HAL 支持（`camera2` 的 supported stream combinations）、后台限制（无前台服务开相机即拒）。指示器（绿点）由 SystemUI 监听 camera use 事件强制点亮——隐私不可关闭。

第三章 编解码与 DRM：双队列模型



图 3 MediaCodec 双队列（input/output buffer 池）、Codec2 组件链、MediaDrm 安全通路与 AAudio

text MediaCodec 形状（读码归纳）

```
App ↔ [inputBuffer 队列] ↔ 编解码器 ↔ [outputBuffer 队列] ↔ App
• configure(format, surface) → start() → dequeue/queue 循环（或异步回调）
• 解码直投 Surface（buffer 不过 App 手）——视频播放省一跳
• 底层：OpenMAX → Codec2（组件链，参数运行时可调）迁移中
MediaDrm（书11 呼应）：
• DRM 会话密钥走 KeyMint 同源硬件路径；Secure Decoder 全链加密
• L1（硬件全链，4K）/L3（软件，受限）安全级
```

AAudio（8+ 音频新世代）：MMAP no-IRQ 硬件时钟直驱，stream builder+状态机；底层仍由 AF/AP 管理新旧并存。三板斧观测：`dumpsys media.{codec,audio_flinger,camera}`。

第四章 设计决策九问

- **为什么音频混音在独立 audioserver 进程？** 崩溃隔离（App 坏歌不断）+ 实时调度统一管理（混音线程 SCHED_FIFO 集中一处）。
- **为什么策略（AP）与执行（AF）分离？** 设备拓扑/音效策略高频定制（厂商各异），混音引擎相对稳定——切分线画在"变化频率"上。
- **为什么焦点是约定？** 强制停流会破坏合法场景（导航+音乐混音）；把最终裁量权给最知情的 App。
- **为什么 Camera2 改成请求流水线？** 逐帧可控才能做 ZSL/HDR+/多摄；黑箱模型封死了这些能力。
- **为什么相机结果带 metadata？** 后处理（人脸/夜景算法）需要"这帧怎么拍的"；只给图等于丢参数。

- **为什么 HAL 版本众多（2.4-2.6+AIDL 并行）？** 相机能力爆炸式演进+厂商碎片化——版本化是无奈也是必需（书1 HAL 三代呼应）。
- **为什么 MediaCodec 用双队列池化？** buffer 分配昂贵（图形内存）；池化+复用把分配从热路径挪走。
- **为什么 DRM 解码要 Secure Decoder？** 高清内容授权要求明文不可见（L1）；与 fscrypt 同一哲学——密文到最后一刻。
- **为什么 AAudio 不替换 AudioTrack？** 生态兼容成本；新旧并存让新 API 只服务低延迟场景（增量演进优于推倒重来）。

附录 A 观测实操

目标	命令	看点
音频全表	dumpsys media.audio_flinger	Track/线程/延迟模式
音频策略	dumpsys media.audio_policy	路由/设备/焦点栈
相机全表	dumpsys media.camera	客户端/会话/in-flight
编解码	dumpsys media.codec	实例/格式/buffer
权限	appops get <pkg> CAMERA	使用记录（绿点数据源）

附录 B 快问快答（14 条）

- AF 与 AP 的分工一句话？——混音车间与调度室。
- createTrack 在哪？——AudioFlinger.cpp:1023。
- 三个延迟档？——normal/fast track/MMAP no-IRQ。
- 焦点是强制的吗？——约定，混音层不拦。
- 插耳机音乐怎么切的？——AP 设备状态机重路由。
- Camera2 核心抽象？——异步请求-结果对。
- connectHelper 干什么？——客户端接入仲裁（:1750）。
- 预览本质？——setRepeatingRequest 无限重复。
- CameraX 是新管线吗？——不是，Camera2 易用层。
- 相机绿点能关吗？——不能（SystemUI 强制）。
- MediaCodec 双队列？——input/output buffer 池化复用。
- Codec2 是什么？——组件链式新一代编解码框架。
- DRM L1/L3 区别？——硬件全链 vs 软件受限。
- AAudio 底层？——仍由 AF/AP 管理（并存非替代）。

附录 C 自检题

- 默画音频四段与 20ms 周期内发生的事。
- 焦点 LOSS 后系统做了什么、没做什么?
- 一次拍照从 capture() 到 onCaptureCompleted 的全链（含 in-flight 语义）。
- MediaCodec 直投 Surface 为什么省一跳?
- DRM L1 与书11 KeyMint 的关系。

第五章 音频框架的前世与生态位

5.1 从 AudioSystem 到 AAudio 的十年

text 音频 API 世代（读码归纳）

```
2011-：AudioTrack/AudioRecord（libaudioclient）
——推拉模型，set/write；至今仍是大多数 App 的底座
2014-：AAudio（8.0，O）
——stream builder+状态机；MMAP no-IRQ 低延迟路径
2017-：SoundPool/MediaPlayer 门面化（内部仍 AudioTrack）
——ExoPlayer/媒体3 统一播放器层（App 侧库，非系统）
系统侧不变：AF 混音 + AP 策略 + HAL
// 讲解注释：API 换代三层但管线一层——"换门面不换车间"，
// 这正是分层的价值：应用层进化不迫害底层重写。
```

5.2 声卡之外的"音频设备"：蓝牙与 USB

路由策略的特殊分支：**蓝牙 A2DP**（音频走协议栈不经声卡，delay reporting 支持音画同步）与 **USB Audio Class**（外接 DAC 直挂 ALSA）。两者都由 AP 统一抽象为"输出设备"——App 无感。排障：“连了蓝牙没声音"先查 AP 设备状态机（`dumpsys media.audio_policy` 的 connected devices），再查 A2DP 栈（书10 呼应：蓝牙也是独立 Mainline 模块）。

附录 D 动手实验室（半天量）

实验一"音频延迟实测"：AudioTrack 写正弦+示波器 App（或 tap-to-tone）测 normal vs PERFORMANCE_MODE_LOW_LATENCY 差值；AAudio MMAP 对比（需 8+ 设备）。

实验二"路由现场"：播放中插拔耳机，`dumpsys media.audio_policy` 前后对比（路由表/活动设备变化）；开勿扰后铃声/媒体音量的策略差异。

实验三"焦点攻防"：两个 App 同时 requestAudioFocus(GAIN)，观察广播次序；一个不守约继续播——验证"混音不停"（第一章 1.2 的实证）。

实验四"in-flight 之旅"：`dumpsys media.camera` 连续采样预览中的 inFlightCount；拍照瞬间 spike（高分辨率请求入队）——第二章流水线实证。

实验五"Offload 省电"：播放本地 MP3 闲置 10 分钟记录耗电，对比"关 offload"（开发者选项）——第一章 1.3 的数字版。

教科书习题集（分级）

第一级：简答题（检验概念理解）

（从附录 B 快问快答中挑 5 题，闭卷手写答案。）

第二级：分析题（检验机制理解）

- **追踪调用链**：画出本册核心机制的完整调用图。
- **失败推演**：核心机制某步失败，系统会怎样?
- **跨册串联**：本册机制与 Binder 十跳（书3）的交汇点。

第三级：设计题

- **架构演进**：重构本册核心机制，你会怎么改?
- **排障方案**：设计分层定位方案。

术语表

（术语表：从附录 B 提取本册专属术语，按拼音排序。）

排障手册：媒体管线问题定位

1. dumpsys 定位状态 → 2. logcat 查看日志 → 3. 深入各章。